other object-relational mapper (ORM) in a standard way, you should be safe. EF Core has built-in protections against SQL injection, so as long as you're not doing anything funky, you should be fine.

SQL injection vulnerabilities occur when you build SQL statements yourself and include dynamic input that an attacker provides, even indirectly. EF Core provides the ability to create raw SQL queries using the `FromSqlRaw()` method, so you must be careful when using this method.

Imagine your recipe app has a search form that lets you search for a recipe by name. If you write the query using LINQ extension methods (as discussed in chapter 12), you would have no risk of SQL injection attacks. However, if you decide to write your SQL query by hand, you open yourself to such a vulnerability, as shown in the following listing.

Listing 29.6 An SQL injection vulnerability in EF Core due to string concatenation

```
public IList<User> FindRecipe(string search)        ❶
{
    return _context.Recipes                         ❷
        .FromSqlRaw("SELECT * FROM Recipes" +       ❸
                 "WHERE Name = '" + search + "'")   ❹
        .ToList();
}
```

❶ The search parameter comes from user input, so it's unsafe.

❷ The current EF Core DbContext is held in the _context field.

❸ You can write queries by hand using the FromSqlRaw extension method.

❹ This introduces the vulnerability—including unsafe content directly in an SQL string.

In this listing, the user input held in `search` is included directly in the SQL query. By crafting malicious input, users can potentially perform any